Bilan de l'application Scrabble

Oct 23, 2026 · @Islain Kouam

En trois jours (21 au 23 octobre 2026), la plupart des points critiques du premier bilan sont réglés : le jeu respecte enfin les règles, et l'application est jouable et installable sur téléphone. Il reste surtout de l'infrastructure (parties stockées en mémoire) et du nettoyage. État vérifié sur `main` : 42 tests backend réussis, build et vérification i18n au vert.

En un coup d'œil

Neuf points sur dix sont passés de cassé à fonctionnel ; seul le lint reste en dette.

Point	Premier bilan (21 oct.)	Aujourd'hui (23 oct.)
Tests backend	19 tests, aucun ne s'exécutait	42 réussis, 1 ignoré (demande une base de données)
CI GitHub Actions	Aucune	Tests + chaîne Alembic, i18n, build ; lint non bloquant
Règles de pose	Trous et mots isolés acceptés	Contiguïté et connexion vérifiées
Calcul du score	Faux : lettres existantes ignorées, 7 cases 2L sur 24	Conforme aux règles, plateau complet
Joker	Injouable (humain, multijoueur, IA)	Fonctionne partout, l'IA s'en sert
Multijoueur	Impossible de poser une lettre	Placement réparé (toucher + bon index)
Mobile	Glisser-déposer seulement, mise en page cassée	PWA installable, écran dédié portrait et paysage, sans défilement
`frontend/.env` versionné	Oui	Non, remplacé par des `.env.example`
Traductions	216 clés	232 clés, parité vérifiée
Lint ESLint	Non mesuré	39 erreurs, toutes antérieures (non bloquant en CI)

Ce qui est solide maintenant

Le moteur de jeu, le frontend et la PWA sont au niveau d'une vraie application.

Moteur de jeu

- Validation complète des coups : alignement, contiguïté, connexion au plateau, case centrale au premier coup.
- Le score compte toutes les lettres de chaque mot formé et applique les bonus uniquement aux nouvelles cases ; le bonus de 50 points pour un scrabble est appliqué.
- Le joker utilise la notation standard : une lettre minuscule veut dire « joker joué comme cette lettre ».
- L'état partagé entre parties a été supprimé : deux parties simultanées ne se faussent plus le score.
- L'IA tourne dans un thread séparé et ne bloque pas le serveur.

Frontend

- Calcul d'aperçu du score : un utilitaire unique, `utils/scoring.js`, au lieu de deux copies divergentes.
- Fenêtre de choix du joker : traduite en français et en anglais, utilisable au clavier sur ordinateur.
- Téléphone en portrait : barre de navigation sur une ligne avec un menu ☰, bandeau des scores, plateau aussi grand que possible, chevalet et actions en bas.
- Téléphone en paysage : plateau à gauche, chevalet et actions à droite.
- Défilement figé pendant une partie, sans rebond ni « tirer pour actualiser ».
- Ordinateur : mise en page inchangée.

PWA

- Manifest, icônes et service worker en place, avec mise à jour automatique à chaque déploiement.
- Service worker désactivé dans l'app Capacitor.

Ce qui reste à faire

Le seul vrai risque en production est le stockage des parties solo en mémoire ; le reste relève du nettoyage.

1. Critique : les parties solo sont stockées en mémoire

Elles vivent dans `GameEngine.active_games`.

- Elles sont perdues à chaque redéploiement ou redémarrage Railway.

- Elles ne sont libérées que sur abandon. Une partie terminée normalement, ou quittée en fermant l'onglet, reste en mémoire indéfiniment : c'est une fuite qui grossit avec le nombre de parties.
- L'API ne peut pas tourner sur plusieurs instances. Le rate limiter de l'auth est aussi en mémoire, avec le même problème.
- Solution : persister les parties en base (table `multiplayer_games` ou équivalente), ou au minimum nettoyer les parties terminées ou inactives.

2. Moyen : dette de qualité

- **Lint** : 39 erreurs ESLint, toutes antérieures. Les principales viennent de règles React 19 (`set-state-in-effect` , `refs` , `only-export-components`). Fichiers les plus touchés : `TutorialOverlay.jsx` , `SettingsContext.jsx` , `TutorialContext.jsx` . Le lint reste non bloquant dans la CI tant que ce n'est pas nettoyé.
- **Bundle de 702 ko** (214 ko compressé), surtout à cause de Firebase. Le découpage du code accélérerait le premier chargement sur mobile.
- **Textes en dur en français**, qui ne changent pas en anglais : « Tour de… » et « Sac » dans `GameHeader` , « Scores / EN COURS » dans `ScorePanel` , « Multijoueur » dans la navigation, et la plupart des textes de `MultiplayerPage` .
- **Pydantic** : 5 modèles utilisent encore `class Config` , ce qui cassera avec Pydantic v3.
- **Règles Firebase RTDB** absentes du dépôt, alors que la sécurité du multijoueur repose dessus.

3. Dispersion mobile

- Le dossier `scrabble_mobile_pwa/` fait doublon avec le frontend, qui est maintenant lui-même une PWA. Il peut être supprimé.
- Le dépôt `scrabble_mobile` (Expo) n'a pas bougé depuis le 24 mai 2026 : ni auth, ni multijoueur, ni i18n, ni corrections de règles. Recommandation : l'archiver et miser sur PWA + Capacitor.

4. À vérifier

- Le multijoueur sur téléphone et le joker en multijoueur n'ont pas pu être testés sans Firebase. Une partie à deux sur de vrais téléphones suffira.

5. Méthode de travail

- Le 21 octobre, un push sur `main` a écrasé 3 commits, réappliqués depuis. Comme Railway et Vercel déploient sans doute depuis `main` , protéger cette branche (PR obligatoire et CI au vert) éviterait que ça se reproduise.

Priorités suggérées

Persister les parties solo passe en premier : c'est le point qu'un recruteur technique relevèrait en lisant `game_logic.py`.

- [ ] Persister les parties solo et nettoyer celles qui sont terminées
- [ ] Traduire les textes restés en dur en français
- [ ] Versionner les règles RTDB et tester le multijoueur sur téléphone
- [ ] Supprimer `scrabble_mobile_pwa/` et archiver le dépôt Expo
- [ ] Nettoyer le lint pour le rendre bloquant dans la CI, puis découper le bundle
- [ ] Migrer les modèles Pydantic vers `ConfigDict`
- [ ] Protéger la branche `main` (PR obligatoire, CI au vert)

Pour un portfolio, le projet a déjà tout ce qu'on attend d'une vraie application : moteur d'IA aux règles correctes, temps réel, i18n, accessibilité, PWA mobile soignée, tests et CI.